Agent Containerization Strategy

OpenSaaS + Value Fabric Architecture

A comprehensive guide to containerizing AI agent services

within the OpenSaaS ecosystem

React / Node.js / PostgreSQL / Docker

May 2026

Table of Contents

Right-click and select "Update Field" to refresh page numbers

Architecture Overview

Current Stack

Your application combines two complementary platforms:

- OpenSaaS (opensaas.sh): A Wasp-based full-stack SaaS template providing React frontend, Node.js backend, PostgreSQL database, authentication, payments, and admin dashboard.
- Value Fabric Architecture (kimi.page): An enterprise AI architecture defining intelligent data ingestion, ontology-guided extraction, knowledge graphs, ground truth layers, and agentic workflows.

The containerization challenge centers on deploying the AI agent services described in your Value Fabric Architecture alongside your existing OpenSaaS application infrastructure.

Agent Ecosystem

Based on your Value Fabric Architecture, the agent ecosystem comprises five specialized service types:

Agent Type	Language	Key Capabilities
Data Ingestion	Python	Web scraping, document parsing, API connectors
Extraction	Python	LLM-based extraction, Pydantic schemas, ontology alignment
Knowledge Graph	Python	Neo4j operations, RDF/OWL, vector similarity, GraphRAG
Ground Truth	Python	Fact verification, provenance tracking, maturity scoring
Workflow	Python	Task orchestration, agent coordination, event routing

Containerization Strategy

Service Architecture

The recommended architecture extends your existing OpenSaaS deployment with a dedicated agent services tier. All services communicate through well-defined APIs and share infrastructure where appropriate.

Tier Structure

Tier	Services	Purpose
Application	wasp-client, wasp-server	User-facing SaaS application
Agents	5 agent services + registry	AI/ML processing pipeline
Data	postgres, neo4j, redis	Structured, graph, and cache storage

Communication Patterns

Services communicate through the following patterns:

0. Synchronous (REST/gRPC): Wasp backend calls agent services for real-time operations.
1. Asynchronous (Message Queue): Agents publish events to Redis/RabbitMQ for decoupled processing.
2. Database-Backed: Agents write results to PostgreSQL; Wasp backend reads for display.
3. File/Volume Sharing: Shared Docker volumes for document processing pipelines.

Docker Compose Configuration

The following Docker Compose structure organizes all services. The OpenSaaS Wasp application (client + server) builds via wasp build and deploys as separate containers.

Core Services

```
version: '3.8'services:  # ========================================== # OpenSaaS Application Tier  # ==========================================  wasp-server:  build:      context: .      dockerfile: .wasp/build/Dockerfile    ports:      - '${SERVER_PORT:-3001}:3001'    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}      JWT_SECRET: ${JWT_SECRET}      WASP_WEB_CLIENT_URL: ${WASP_WEB_CLIENT_URL}      REDIS_URL: redis://redis:6379    depends_on:      postgres:        condition: service_healthy      redis:        condition: service_started    networks:      - app-network  wasp-client:    build:      context: .      dockerfile: .wasp/build/web-app/Dockerfile    ports:      - '${CLIENT_PORT:-3000}:3000'    environment:      REACT_APP_API_URL: ${WASP_SERVER_URL}    depends_on:      - wasp-server    networks:      - app-network
```

Agent Services

```
  # ========================================  # Agent Services Tier  # ========================================  agent-ingestion:    build: ./agents/ingestion    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}      REDIS_URL: redis://redis:6379      KIMI_API_KEY: ${KIMI_API_KEY}    volumes:      - ingestion-data:/app/data    depends_on:      - postgres      - redis      - neo4j    networks:      - app-network  agent-extraction:    build: ./agents/extraction    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}      REDIS_URL: redis://redis:6379      KIMI_API_KEY: ${KIMI_API_KEY}    depends_on:      - postgres      - redis      - neo4j    networks:      - app-network  agent-knowledge-graph:    build: ./agents/knowledge-graph    environment:      NEO4J_URI: bolt://neo4j:7687      NEO4J_USER: ${NEO4J_USER}      NEO4J_PASSWORD: ${NEO4J_PASSWORD}      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}    depends_on:      - neo4j      - postgres    networks:      - app-network  agent-ground-truth:    build: ./agents/ground-truth    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}      REDIS_URL: redis://redis:6379    depends_on:      - postgres      - redis    networks:      - app-network  agent-workflow:    build: ./agents/workflow    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}      REDIS_URL: redis://redis:6379      AGENT_REGISTRY_URL: http://agent-registry:8080    depends_on:      - postgres      - redis      - agent-ingestion      - agent-extraction      - agent-knowledge-graph      - agent-ground-truth    networks:      - app-network
```

Infrastructure Services

```
  # ========================================  # Infrastructure Tier  # ========================================  postgres:    image: postgres:16-alpine    environment:      POSTGRES_USER: ${DB_USER}      POSTGRES_PASSWORD: ${DB_PASS}      POSTGRES_DB: ${DB_NAME}    volumes:      - postgres-data:/var/lib/postgresql/data    healthcheck:      test: ['CMD-SHELL', 'pg_isready -U ${DB_USER} -d ${DB_NAME}']      interval: 5s      timeout: 5s      retries: 5    ports:      - '${DB_PORT:-5432}:5432'    networks:      - app-network  neo4j:    image: neo4j:5-community    environment:      NEO4J_AUTH: ${NEO4J_USER}/${NEO4J_PASSWORD}      NEO4J_PLUGINS: '[]'    volumes:      - neo4j-data:/data      - neo4j-logs:/logs    ports:      - '7474:7474'      - '7687:7687'    networks:      - app-network  redis:    image: redis:7-alpine    command: redis-server --appendonly yes    volumes:      - redis-data:/data    ports:      - '6379:6379'    networks:      - app-network  # Optional: Agent service registry and monitoring  agent-registry:    build: ./services/registry    ports:      - '8080:8080'    environment:      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}    depends_on:      - postgres    networks:      - app-networkvolumes:  postgres-data:  neo4j-data:  neo4j-logs:  redis-data:  ingestion-data:networks:  app-network:    driver: bridge
```


Agent Service Specifications

Data Ingestion Agent

Service	agent-ingestion
Stack	Python 3.11 + FastAPI
Port	3002
Capabilities	Web scraping, document parsing, API connectors, file upload handling

The ingestion agent handles all data intake operations. It processes web sources, uploaded documents, and API feeds, then normalizes content for downstream extraction.

Extraction Agent

Service	agent-extraction
Stack	Python 3.11 + FastAPI
Port	3003
Capabilities	Pydantic schema validation, LLM-based entity extraction, ontology alignment

The extraction agent performs ontology-guided information extraction. It uses multimodal LLMs with visual DOM understanding and enforces schema constraints via Pydantic-typed JSON output.

Knowledge Graph Agent

Service	agent-knowledge-graph
Stack	Python 3.11 + Neo4j Driver
Port	3004
Capabilities	RDF/OWL serialization, graph traversal, vector similarity, GraphRAG

The knowledge graph agent manages the Neo4j-based enterprise knowledge graph. It handles RDF triple serialization, semantic alignment via vector similarity, and hybrid retrieval for multi-hop reasoning.

Ground Truth Agent

Service	agent-ground-truth
Stack	Python 3.11 + FastAPI
Port	3005
Capabilities	Fact verification, provenance tracking, maturity scoring, human-in-the-loop

The ground truth agent maintains the evidence-backed fact store. It implements multi-stage verification pipelines, tracks data freshness, and manages the maturity ladder from raw (Level 0) to operationalized (Level 5).

Workflow Orchestrator

Service	agent-workflow
Stack	Python 3.11 + Celery
Port	3006
Capabilities	Task scheduling, agent coordination, event-driven workflows, retry logic

The workflow orchestrator coordinates multi-agent pipelines. It manages the normalized value tree (value drivers, personas, use cases, capabilities) and routes tasks between specialized agents.

Deployment Options

Development

For local development, use the complete Docker Compose stack:

```
# Start all servicesdocker-compose up --build# Or start specific
servicesdocker-compose up wasp-server wasp-client postgres redis
```

The Wasp CLI remains useful for development workflows:

```
# In development, use Wasp's built-in commandswasp db start          # Starts
PostgreSQL via Dockerwasp db migrate-dev    # Run migrationswasp start
# Start dev server
```

Production

For production, the recommended approach depends on your infrastructure:

Option A: Wasp Deploy (Fly.io/Railway) + Separate Agent Host

- Deploy OpenSaaS client/server via wasp deploy fly launch
- Deploy agent services on a dedicated VM or container platform
- Connect via environment variables and API keys

Option B: Full Docker Compose Deployment

- Use the complete docker-compose.yml on a VPS or dedicated server
- Manage via Docker Swarm or Kubernetes for orchestration
- Use Traefik or nginx as reverse proxy with SSL termination

Option C: Hybrid Cloud

- OpenSaaS on managed platform (Fly.io, Railway, Render)
- Agent services on GPU-enabled instances (for LLM inference)
- Database on managed PostgreSQL (Supabase, Neon, AWS RDS)

Environment Configuration

Create a .env file for local development and use your platform's secret management for production:

```
#
DatabaseDB_USER=postgresDB_PASS=secure_password_hereDB_NAME=opensaas_agentsDB_PORT=
5432#
OpenSaaSJWT_SECRET=your_jwt_secret_min_32_charsWASP_SERVER_URL=http://localhost:300
1WASP_WEB_CLIENT_URL=http://localhost:3000SERVER_PORT=3001CLIENT_PORT=3000#
Neo4jNEO4J_USER=neo4jNEO4J_PASSWORD=secure_neo4j_password#
RedisREDIS_URL=redis://redis:6379# Kimi APIKIMI_API_KEY=your_kimi_api_key# Payment
Providers (OpenSaaS)STRIPE_API_KEY=sk_test_...STRIPE_WEBHOOK_SECRET=whsec_...# AWS
S3 (if used for file
uploads)AWS_S3_IAM_ACCESS_KEY=...AWS_S3_IAM_SECRET_KEY=...AWS_S3_FILES_BUCKET=...AW
S_S3_REGION=us-east-1
```

Security Considerations

- Network Isolation: All services communicate through the app-network bridge. No external exposure except wasp-client (port 3000) and wasp-server API (port 3001).
- Secret Management: Use Docker secrets or platform-native secret management. Never commit .env files.

- Database Security: PostgreSQL and Neo4j only expose ports internally. Use strong passwords and enable SSL in production.
- API Authentication: Agent services should validate JWT tokens from the Wasp backend or use service-to-service API keys.
- Rate Limiting: Implement rate limiting on agent APIs to prevent abuse.
- Input Validation: All agent inputs must pass Pydantic schema validation before processing.

Operational Best Practices

Health Checks

Implement health check endpoints on all agent services. Docker Compose depends_on with condition: service_healthy ensures proper startup order.

Logging

Use structured JSON logging. Centralize logs with the ELK stack or Grafana Loki for production deployments.

Monitoring

```
# Example Prometheus metrics endpointfrom prometheus_client import Counter,
HistogramREQUEST_COUNT = Counter('agent_requests_total', 'Total requests',
['agent', 'status'])REQUEST_DURATION = Histogram('agent_request_duration_seconds',
'Request duration', ['agent'])
```

Scaling

- Stateless agents (ingestion, extraction) scale horizontally with multiple replicas.
- Database writes should use connection pooling (PgBouncer).
- Neo4j read replicas can handle graph query load.
- Use Redis for caching frequent queries and session state.

Backup Strategy

- PostgreSQL: Daily automated backups via pg_dump or managed service snapshots.
- Neo4j: Regular backups via neo4j-admin dump.
- Redis: Enable AOF persistence for data durability.